Win x64 Shellcode – Part 1: Why We Can't Use Static Addresses

April 10, 2026 · 5 min

I have a background in writing 32-bit shellcodes for Windows [1][2][3]. The transition to x64 looks like a cosmetic change at first glance (wider registers, a different calling convention). In reality, it is an entirely different mental model.

The goal of this series is to describe the fundamentals of Win64 shellcode development in a simplified yet comprehensive way. It does not aim to be exhaustive documentation on the topic, but rather to provide accessible learning material that can serve as a stepping stone into this field.

We will start with what most tutorials skip, whether intentionally or not (why things that worked on x86 don’t work on x64, and how to deal with that).

If you have ever seen a simple Linux x86 shellcode, it likely contained immediate values like `mov eax, 0xb` or absolute addresses written directly into registers. On Windows x64, that doesn’t work – and this article explains why.

Prerequisites

The series assumes the reader has a basic understanding of x86/x64 assembly (registers, calling conventions, and common instructions) and a general idea of what a process is and how the stack works.

What Is Position-Independent Code

When talking about shellcodes, we need to get familiar with the concept of Position-Independent Code. A shellcode is executable code that we must deliver and execute inside a target process, without knowing in advance at what address it will be placed. We cannot assume the shellcode will be stored at `0x00401000` or any other fixed address. In other words, we must accept that our shellcode will almost certainly end up at a different address than we originally intended. Therefore, the shellcode must be capable of running at any address in memory.

Code that works correctly regardless of where in memory it resides is called **Position-Independent Code (PIC)**.

On Windows x64, things are even more complex, because we need to call Windows API functions whose addresses are not known to us in advance. Linux shellcodes have it easier in this regard.

Why We Don’t Know Windows API Function Addresses

On older versions of Windows, life was much simpler for shellcode authors. Because DLLs always loaded at the same address, all one had to do was look up the address of a given Windows API function (such as `WinExec` [6] or `CreateProcess` [7]) for a specific Windows version and hardcode it directly into the shellcode. This guaranteed the shellcode would work reliably on that version of Windows. Microsoft was aware of these weaknesses and introduced several mitigations that significantly complicated or entirely prevented this approach.

ASLR – Address Space Layout Randomization

Since Windows Vista, Microsoft has employed **ASLR**[8]. ASLR fundamentally changed how processes work. DLLs no longer load at predictable addresses; instead, their placement in memory is randomized. This means that on every system boot, key modules such as `ntdll.dll`, `kernel32.dll`, or `kernelbase.dll` are loaded at a random base address.

```
First boot:
  kernel32.dll  ->  0x7FFB`A3C0`0000

Second boot:
  kernel32.dll  ->  0x7FFB`C1D0`0000

Third boot:
  kernel32.dll  ->  0x7FFB`8F40`0000
```

It follows that the address of `kernel32.dll`, as well as the address of its exported function `WinExec`, changes with every restart. A shellcode with a hardcoded address would work at most once, on a specific system in a specific session.

DLL Rebasing

The situation is even more complex. Even within a single session, the same module may be loaded at a different address in different processes if an address collision occurs with another module. In such cases, DLL rebasing kicks in. This is the process of adjusting a DLL's preferred base address – the address stored in the DLL's headers that indicates where it should be loaded. If that address is already occupied by another module, rebasing is performed. ASLR only amplifies this effect.

How Does a Shellcode Resolve Function Addresses?

Despite all the obstacles described above, there is a way to obtain these dynamic addresses at runtime within a running process. The solution relies on structures that Windows maintains directly in the memory of every process, without requiring any external dependencies.

Two structures are key:

TEB (Thread Environment Block)[4] – maintained for every thread in a process and accessible via the `GS` segment register on x64. It contains a pointer to the PEB structure.

PEB (Process Environment Block)[5] – maintained for every process. Among other things, it contains a list of all loaded modules along with their current base addresses.

Simply put: even with ASLR, a shellcode has runtime access to the current addresses of all loaded modules. You just need to know where to look.

```
GS:[0x60]  ->  PEB
                 └── Ldr
                      └── InMemoryOrderModuleList
                            ├── ntdll.dll        (current base address)
                            ├── kernel32.dll     (current base address)
                            └── ...
```

Summary

- A shellcode must be position-independent – it cannot assume where in memory it will be placed.
- Windows API functions have randomized addresses at every system or process start, thanks to ASLR.
- Hardcoded addresses in a shellcode are therefore a non-viable approach.
- The solution is runtime resolution via the TEB and PEB structures, which are available within every process.

What's Coming in the Next Part

We will examine the PEB structure and how Windows maintains the list of loaded modules. We will walk through the path from `GS:[0x60]` to the base address of `kernel32.dll` – step by step, with real offsets.

References

Author’s Shellcodes

- [1] Exploit-DB #13828
- [2] Exploit-DB #17323
- [3] Exploit-DB #24318

Windows Structures

- [4] TEB (Thread Environment Block) – Microsoft Learn
- [5] PEB (Process Environment Block) – Microsoft Learn

Windows API Functions

- [6] WinExec – Microsoft Learn
- [7] CreateProcess – Microsoft Learn

Security Mitigations

- [8] ASLR – Exploit protection reference – Microsoft Learn

Shellcode | Windows | X64 | Assembly | Exploit-Dev | Windows-Internals

NEXT »
Win x64 Shellcode – Part 2: TEB, PEB and List of Loaded Modules

© 2026 proteqtum · Powered by Hugo & PaperMod